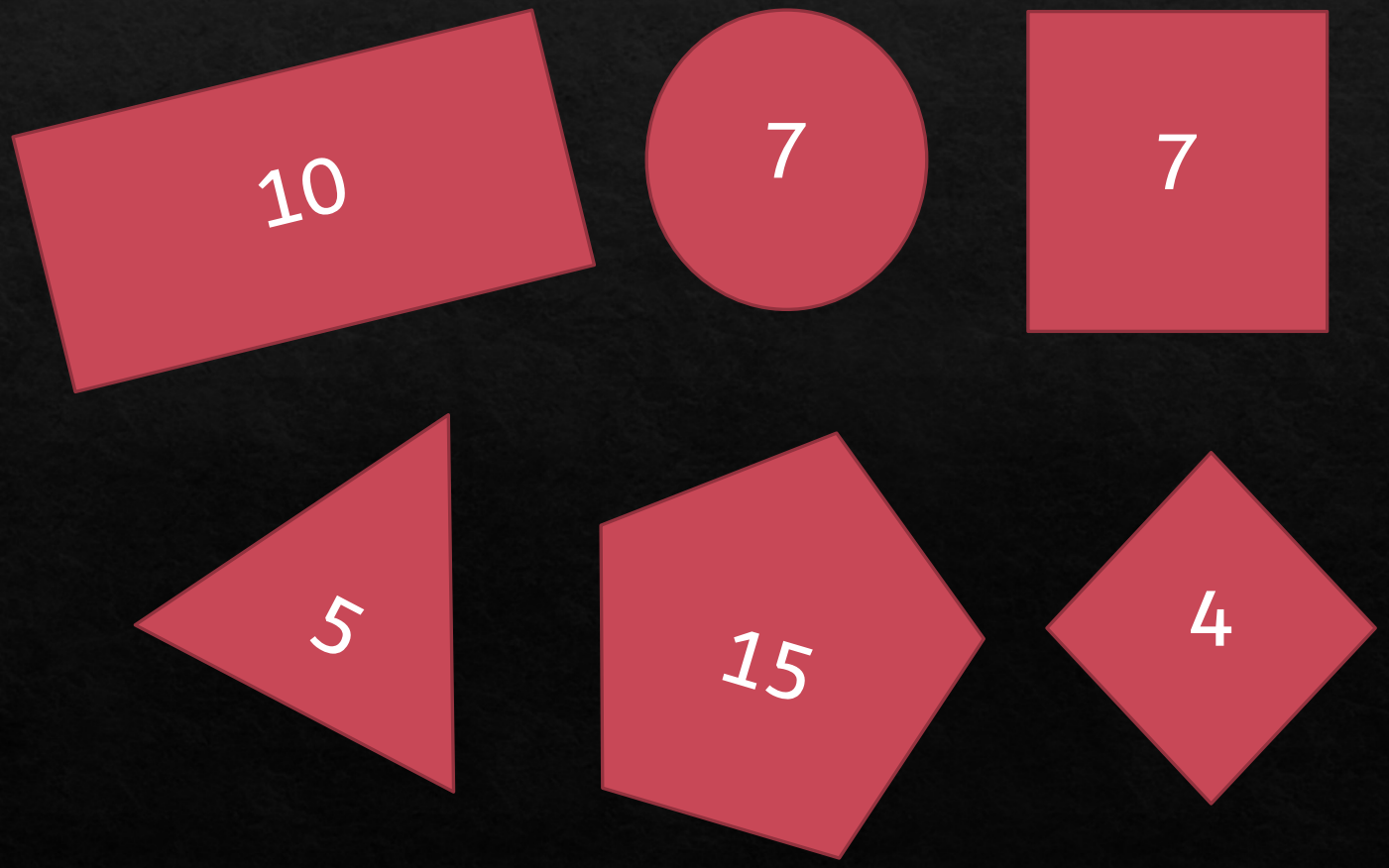
# Жадные алгоритмы

максимальная суммарная площадь, размен монет, выбор заявок, задача о рюкзаке

# Общие сведения

# Максимальная площадь

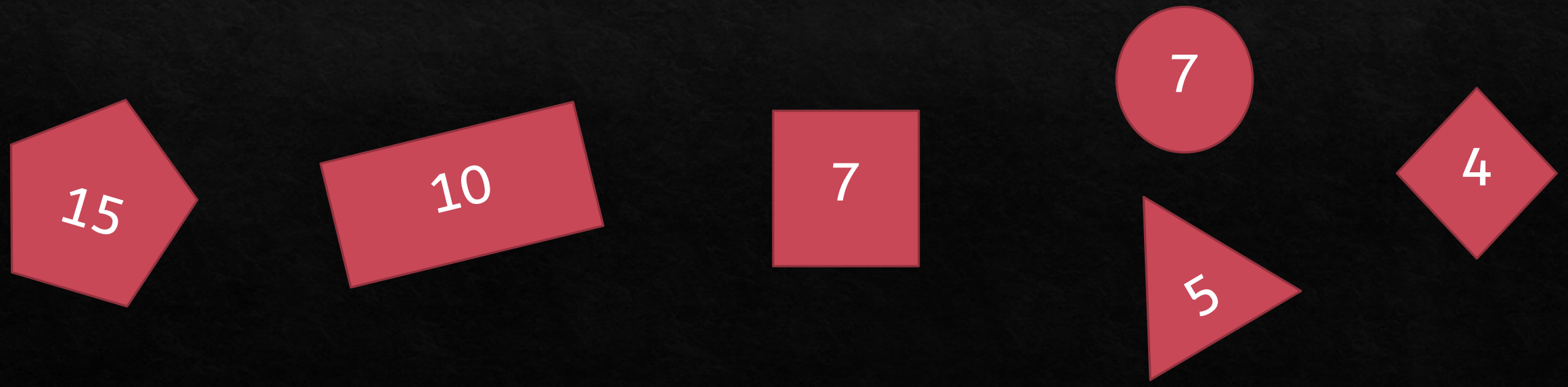
Есть набор из некоторого количества фигур, нужно выбрать 3 из них таким образом, чтобы суммарная площадь была максимальной.



# Оптимизация и жадный алгоритм

Задачи оптимизации требуют поиска экстремума (минимума или максимума) при некоторых ограничениях. Жадные алгоритмы являются одним из методов решения задач оптимизации.

Идея жадного алгоритма: на каждом шаге мы будем делать «локально оптимальный» выбор пока не получим глобально оптимальное решение.



# Принцип оптимальности решения для подзадач

Рассмотрим частный случай задачи оптимизации, когда нужно из некоторого множества *B* выбрать подмножество *A*, которое даёт оптимальное решение при заданных ограничениях. Тогда принцип оптимальности состоит в том, что последовательность локально оптимальных выборов даёт решение для всех подзадач.

Пусть $A = \{a_1, a_2, \dots, a_n\}$ − оптимальное решение для задачи из $N$ элементов на множестве $B$, тогда $\acute{A} = \{a_2, \dots, a_n\}$ − оптимальное решение для задачи из $N - 1$ элементов на множетсве $\acute{B} = B \backslash \{a_1\}$

# Выбор заявок

# Формулировка

Для работы на суперкомпьютере поданы заявки на проведение вычислений. Для каждого запуска определено время включения и время выключения. Необходимо выбрать заявки таким образом, чтобы их количество было максимальным.

| | **1** | **2** | **3** | **4** | **5** | **6** | **7** | **8** |
|---|---|---|---|---|---|---|---|---|
| начало | 1:00 | 1:00 | 2:00 | 4:00 | 5:00 | 7:00 | 8:00 | 9:00 |
| конец | 8:00 | 5:00 | 3:00 | 7:00 | 9:00 | 16:00 | 10:00 | 11:00 |

# Алгоритм

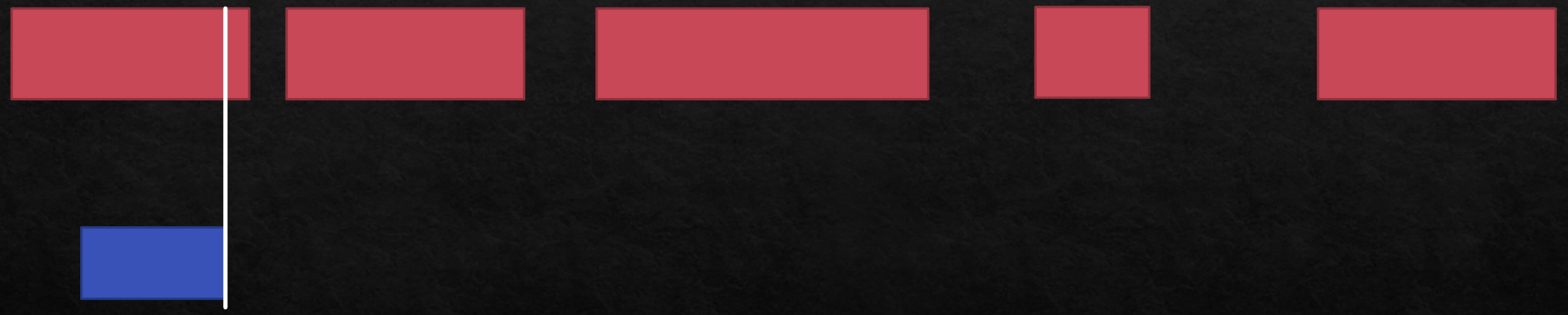
На каждом шаге будем выбирать такую заявку, которая заканчивается раньше, что позволит нам оставить больше времени для других заявок. Предполагается, что мы получим оптимальное решение.

| | **3** | **2** | **4** | **1** | **5** | **7** | **8** | **6** |
|---|---|---|---|---|---|---|---|---|
| начало | 2:00 | 1:00 | 4:00 | 1:00 | 5:00 | 8:00 | 9:00 | 7:00 |
| конец | 3:00 | 5:00 | 7:00 | 8:00 | 9:00 | 10:00 | 11:00 | 16:00 |

# Идея доказательства

Если существует оптимальное решение, то можно заменить первую по времени заявку на заявку, которая заканчивается первой.

Оптимальное решение.



Оставшегося времени, очевидно, хватит для размещения оставшихся заявок.

Применяем рассуждения по индукции.

Заметим, что выполняется принцип оптимальности подзадачи

# Задача о размене

# Формулировка

Дан набор монет с различным номиналом, например 1, 5, 25, 50, 100, нужно набрать некоторую заданную сумму P с использованием наименьшего числа монет. Количество монет каждого номинала не ограничено.

# Алгоритм

На каждом шаге будем выбирать самое большое возможное количество монет с наибольшим номиналом. Когда монету с этим номиналом не удаётся использовать переходим к меньшему номиналу и повторяем действие.

| | |
|---|---|
| 73 | Нельзя использовать ни одной монеты по 100. |
| 23 | Использовали 1 монету по 50. |
| 23 | Нельзя использовать ни одной монеты по 25. |
| 3 | Использовали 4 монеты по 5. |
| 0 | Использовали 3 монеты по 1. |

# Идея доказательства

Если система монет устроена таким образом, что каждый последующий номинал кратен предыдущему, *каноническая система*, то жадный алгоритм работает.

Пусть $\{a_1, a_2 \dots a_n\}$ $-$ набор монет различного кратного номинала от меньшего к большему

Пусть $P$ $-$ сумма, которую необходимо набрать, причём $P > a_n$

Пусть $\{b_{1,} \dots b_n\}$ $-$ оптимальный набор.

Предположим, что $b_n \equiv 0$. Покажем, что набор не может давать $P$.

Очевидно, что $b_i \leq \dfrac{a_{i+1}}{a_i} - 1$

$$a_1 b_1 + a_2 b_2 + \dots + a_{n-1} b_{n-1} \leq a_1 \left(\frac{a_2}{a_1} - 1\right) + \dots + a_{n-1} \left(\frac{a_n}{a_{n-1}} - 1\right)$$

$a_n - a_1 < a_n < P$ $-$ что противоречит изначальному предположению

# Неканоническая система

Если система монет неканоническая, то жадный алгоритм может не давать верного ответа.

Рассмотрим систему момент 1, 5, 7 и попытается набрать 24 единицы.
Жадный алгоритм даёт 7x3 + 1x3 – 6 монет.
Но 24 можно набрать за 4 монеты: 7x2 + 5x2.

# Задача о рюкзаке

# Формулировка

Дан набор некоторых предметов. Каждый из них имеет определённый уровень полезности, например стоимость, и определённую нагрузку, например вес. Требуется выбрать предметы с наибольшей общей полезностью, не превысив нагрузку.

# Алгоритм

Рассмотрим для каждого элемента из набора величину «удельной полезности», отношение полезности к нагрузке. Будем набирать предметы таким образом: сначала выбираем предметы с наибольшей удельной полезностью, когда нагрузка больше не позволяет разместить предметы, мы переходим к следующему по уровню удельной полезности предмету.

# Корректность для «разделяемых предметов»

Если мы можем брать не цельные вещи, а их части (например предметы представляют собой слитки металла), то алгоритм выдаёт оптимальное решение. Задача имеет название «дробного рюкзака» (fractional knapsack).

Корректность для дробного рюкзака очевидна. Разделим все предметы на части равные единице нагрузки. Тогда мы будем иметь большой набор вещей одинаковой нагрузки, но разной полезности. Задача сводится к выбору максимальной суммы полезности предметов при ограничении их количества.

# Задача рюкзака 0-1

Если предметы можно брать только полностью, то жадный алгоритм может давать не оптимальное решение.

| | | | |
|---|---|---|---|
| 1 | 3 | 6 | 0.5 |
| 2 | 3 | 10 | 0.3 |
| 3 | 4 | 6 | 0.6(6) |
| 4 | 2 | 5 | 0.4 |
| 5 | 2 | 5 | 0.4 |

Вместимость 16.

Жадный алгоритм: 3(4) + 1(3) – полная полезность 7
Оптимальное решение: 3(4) + 4(2) + 5(2) - 8